National Fraud Prevention Challenge

Phase 2: Solution Report

Team: DMJ.ONE

Date: March 16, 2026

Challenge: RBI Innovation Hub & IIT Delhi

Executive Summary

We developed a gradient-boosted tree ensemble for mule account detection achieving AUC-ROC 0.968 (public) and 0.956 (private). Our pipeline processes 16GB of transaction data through 208 engineered features, multi-round confident learning for label noise, and 3-seed cross-validated training with rank-averaged ensembling. We identified and analyzed all 7 red herring categories, achieving near-perfect avoidance on 6 of 7.

1. Approach: Methodology and Model Architecture

1.1 Pipeline Overview

Our end-to-end pipeline consists of 5 stages: (1) Feature engineering across 4 data passes, (2) Label cleaning via confident learning + heuristic noise detection, (3) 3-model ensemble training with multi-seed CV, (4) Temporal window prediction for suspicious activity periods, and (5) Submission generation with calibrated probabilities.

1.2 Feature Engineering (208 features)

Features are computed in 4 memory-efficient passes over the 16GB Parquet dataset (160K accounts, ~400M transactions):

Pass	Category	Count	Key Features
1	Txn Core	~100	Volume, amounts, temporal histograms, channels, MCC, counterparties
2	Txn Extended	~40	Geo spread, IP diversity, balance stats, txn types, dormancy
3	Static	~35	Account age, freeze history, demographics, branch, products
4	Graph	~33	PageRank, HITS, betweenness, Louvain communities, clustering

1.3 Label Cleaning and Noise-Robust Training

- **Confident Learning (2 rounds):** Out-of-fold LightGBM probability estimation with per-class thresholds (Northcutt et al. 2021). Identifies samples where model confidence conflicts with labels.
- **Heuristic Noise Scoring:** Rule-based detection targeting 7 red herring categories using label metadata (alert_reason, mule_flag_date, flagged_by_branch).
- **Combined Weights:** max(CL score, heuristic score) mapped to sample weights in [0.2, 1.0]. Noisy samples are downweighted but not removed, preserving training set size.

1.4 Model Training

- **Ensemble:** LightGBM + XGBoost + CatBoost, each trained independently
- **Cross-validation:** 3-seed (42, 43, 44) x 5-fold stratified CV for prediction stability
- **Encoding:** Frequency encoding for categorical features. No label leakage possible since we only count occurrences, not use target statistics.
- **Hyperparameters:** Optuna-optimized with 100 trials per model. Best parameters use near-zero regularization (alpha ~4e-5, lambda ~8e-7), suggesting clean features.
- **Ensemble method:** Rank averaging (converting predictions to percentile ranks before averaging) outperformed simple averaging, AUC-weighted averaging, and stacked meta-learning.
- **Class imbalance:** scale_pos_weight = 34 (matching ~2.8% mule ratio in training)

1.5 Temporal Window Prediction

For accounts predicted as mules (probability >= threshold), we analyze their full transaction history to identify suspicious activity windows. A sliding window approach scores each time segment by anomaly density (unusual amounts, frequencies, counterparties) and selects the window with the highest concentration of suspicious patterns.

2. Key Findings: Mule Account Patterns

2.1 Top Discriminative Features (SHAP Analysis)

SHAP (SHapley Additive exPlanations) analysis across all three models revealed consistent feature importance patterns:

- **Transaction volume:** Total transaction count and unique counterparties are the strongest signals. Mule accounts show characteristic high-volume, high-counterparty patterns.
- **Graph centrality:** PageRank and betweenness centrality capture mules as intermediaries in transaction networks, acting as bridges between otherwise disconnected account clusters.
- **Channel diversity:** Mules tend to use more diverse transaction channels (ATM, online, POS, wire) compared to legitimate accounts with consistent channel preferences.
- **Temporal patterns:** Mules exhibit more activity during non-business hours and weekends, with less regular temporal distributions compared to legitimate accounts.
- **Structuring indicators:** Transactions structured just below common reporting thresholds are a strong indicator of deliberate evasion behavior characteristic of mule operations.
- **Balance behavior:** Rapid credit-debit cycles where funds pass through quickly with minimal balance retention (pass-through behavior).

2.2 Mule Behavioral Archetypes

We identified several distinct mule behavior patterns in the data:

- **Pass-through mules:** High velocity of credits followed by rapid debits within hours/days. Minimal balance retention. The account serves purely as a transit point.
- **Network hubs:** High fan-in/fan-out with many unique counterparties. Acts as a money funnel, aggregating from many sources and distributing to many destinations.
- **Dormant-burst:** Long periods of inactivity (weeks/months) followed by sudden intense transaction bursts, suggesting activation for specific money laundering operations.
- **Structuring mules:** Deliberately fragmenting transaction amounts to stay below monitoring thresholds. Often combined with multiple channels to further obscure patterns.

3. Experiments: What Worked and What Did Not

Version	Public AUC	Outcome
V1: Baseline (LGB+XGB+CB, target enc)	0.956	Good starting point
V2: Optuna HPO (100 trials/model)	0.956	Found near-zero regularization
V3: Freq enc + rank avg + multi-seed	0.968	BEST. No leakage, stable.
V5: Feature interactions (26 derived)	0.963	Hurt. Trees find interactions.
V6: Pseudo-labeling (2-stage)	0.787	Catastrophic. Diluted signal.
V7: Drop all branch features	0.959	Fixed RH7, destroyed AUC.
V8: Drop branch_code freq/count only	0.958	Surgical fix. Still too costly.

Key Experimental Insights

- **Frequency encoding:** Switching from target encoding to frequency encoding eliminated potential label leakage and

improved public AUC from 0.956 to 0.968 (+1.2%).

- **Multi-seed averaging:** Averaging predictions across 3 random seeds reduced variance and improved ensemble stability without additional compute cost.
- **Feature interactions:** Adding 26 manually crafted feature interactions (flow ratios, density metrics) actually hurt performance. With near-zero regularization, trees already discover optimal splits; explicit interactions introduce redundancy that degrades generalization.
- **Pseudo-labeling:** Two-stage pseudo-labeling catastrophically failed (0.787 AUC). Adding 63K pseudo-labeled samples diluted the mule signal from 2.8% to 1.8%, reinforcing the model's biases rather than correcting them.

Key Lesson: With near-zero regularization and powerful tree models, the simplest feature set produces the best generalization. Feature engineering quality matters more than model complexity or training data augmentation.

4. Results: Performance Metrics

Metric	Public Phase	Private Phase
AUC-ROC (primary ranking)	0.968136	0.955815
F1 Score (best threshold)	0.576	0.536
Temporal IoU	0.184	0.189
OOF CV AUC (3-seed avg)	~0.960	-

The public-to-private gap (~1.2% AUC) indicates reasonable generalization. The private test set contains more subtle mule patterns with attenuated signals, explaining the expected degradation. The consistent OOF CV AUC (~0.960) closely tracks the public score, confirming our cross-validation setup is well-calibrated.

F1 Score optimization: We sweep 100 thresholds from 0.00 to 1.00 and report the maximum F1. Our best threshold is approximately 0.3, reflecting the class imbalance (~2.8% mule ratio) where recall is prioritized over precision.

Temporal IoU: Our sliding window approach achieves coverage on ~960 of the true mule accounts with temporal windows. The modest IoU (~0.18) suggests room for improvement in window boundary precision, though the detection rate is solid.

5. Red Herring Analysis

The challenge includes 7 categories of red herrings designed to mislead models. Our label cleaning pipeline uses heuristic noise scoring to detect and downweight these patterns during training:

RH#	Description	Detection Method	Score
1	Routine Investigation FPs	Heuristic weight 0.6 for alert_reason	0.995
2	Missing alert_reason	Heuristic weight 0.8 for null/empty	0.990
3	Future mule_flag_date	Dates after Jun 2025 downweighted	0.993
4	Very old flag dates	Dates before Jul 2020 downweighted	0.978
5	Boundary date artifacts	Exact boundary dates flagged	0.993
6	Frozen accounts as mules	Null flagged_by_branch detected	0.999
7	Flagged by own branch	Branch features carry signal + noise	0.000

RH7 Deep Dive: The Branch Feature Dilemma

Red Herring #7 (accounts flagged by their own branch) presented the most challenging tradeoff. Our experiments revealed:

- **Baseline:** V3 (all features): Private AUC 0.956, RH7 = 0.000. The model completely falls for this red herring because branch features encode the spurious same-branch pattern.
- **Full removal:** V7 (no branch features): Private AUC 0.893, RH7 = 1.000. Perfect RH7 avoidance but catastrophic AUC drop (-6.3%). All other RH scores also collapsed (0.25-0.74).
- **Surgical removal:** V8 (drop branch_code freq/count only): Private AUC 0.868, RH7 = 1.000. Even surgical removal of just 2 features caused significant damage.

Conclusion: Branch features carry genuine discriminative signal that outweighs the RH7 penalty. The branch_code frequency encoding captures bank network structure that is predictive of mule behavior. Decorrelating the RH7 signal from branch features without losing this predictive power remains an open challenge. A potential approach would be adversarial debiasing during training, but this was not feasible within the competition timeframe.

6. Infrastructure and Reproducibility

- **Compute:** GCP n2-highmem-8 (8 vCPU, 64GB RAM), us-central1-a
- **Training time:** ~12 minutes for full 3-seed x 5-fold CV pipeline
- **Feature engineering:** ~10 minutes for 208 features across 160K accounts and ~400M transactions
- **Memory:** Batch processing with explicit garbage collection for 16GB dataset
- **Stack:** Python 3.10+, LightGBM, XGBoost, CatBoost, scikit-learn, NetworkX, Optuna, SHAP
- **Reproduce:** python run_v3.py --skip-optuna (uses cached Optuna params)

End of Report